

Чат-бот выдумал политику — платит компания.



Дело *Moffatt v. Air Canada* · трибунал · 14.02.2024

- 1 Пассажир спросил чат-бота про тариф по случаю утраты близкого
- 2 Бот: «купи по полной цене, верни разницу в течение 90 дней»
- 3 Реальная политика этого не допускала — и была на той же странице, на которую бот ссылался
- 4 Трибунал: «бот — не отдельное юр. лицо» → компания компенсирует разницу



ЧТО ВЫБРАЛИ

Генеративный чат — архитектура, которая по своей природе сочиняет правдоподобный текст



ЧТО БЫЛО НУЖНО

Детерминированный lookup фиксированной политики — статическая страница или таблица правил

Это не сбой модели. Это неправильный выбор архитектуры под задачу — об этом классе ошибок вся лекция.

ЛЕКЦИЯ 3

Архитектуры AI-систем: агенты, RAG, API

03

Какую архитектуру выбрать под задачу —
и когда правильный ответ «не ИИ»

Раздел 0
Открытие

Раздел 1
Промпт

Раздел 2
RAG

Раздел 3
Fine-tune

Раздел 4
API · агенты

Раздел 5
Фреймворк

Что мы переносим из Лекции 2 — и что надстроим.

Вокруг одного вызова модели надстраиваются 4 обвязки — RAG стоит прямо на эмбедингах Л2.



RAG = semantic search из Л2 §2 + LLM сверху. Single-shot и semantic search не переобъясняем — это готовые блоки. Детали — Раздел 2.

У меня есть задача и доступ к LLM. Какую архитектуру выбрать — и когда правильный ответ «не ИИ»?



Лестница — карта лекции, не требование понять всё сейчас. Каждую ступень разберём отдельно.

РАЗДЕЛ 1

Промпт и его границы

Лестницу мы увидели целиком — теперь снизу: что умеет один вызов и где его потолок, прежде чем что-либо усложнять.



Раздел 0
Открытие

Раздел 1
Промпт

Раздел 2
RAG

Раздел 3
Fine-tune

Раздел 4
API · агенты

Раздел 5
Фреймворк

Дефолт — один вызов с хорошим промптом.

Любой подъём по лестнице оплачивается требованием задачи — это бремя доказательства.



Один вызов LLM с хорошим промптом

- минимальная стоимость (один проход)
- минимальная латентность (нет лишних обращений)
- максимальная предсказуемость (нет петель, нет retrieval, который тихо деградирует)

Добавляешь RAG →

конвейер индексации + векторное хранилище + компонент retrieval (может молча деградировать) + метрики его качества

Добавляешь инструменты / цикл →

внешние вызовы (падают, тормозят) + петли (расходятся) + недетерминизм траектории + новая поверхность атаки

Не усложняй архитектуру без причины, выраженной в требованиях задачи. Это распределение бремени доказательства, а не примитивизм.

Chain-of-thought (пошаговое рассуждение).

CoT — попросить модель не отвечать сразу, а сначала рассуждать по шагам. Технически это всё ещё один вызов — изменён только промпт.

Без CoT

Задача: «Было 23 яблока, 7 испортились, докупили 2 ящика по 6. Сколько хороших?»

→ модель выдаёт правдоподобное, но неверное число

С CoT («решай по шагам»)

$$23 - 7 = 16$$

$$2 \times 6 = 12$$

$$16 + 12 = 28$$

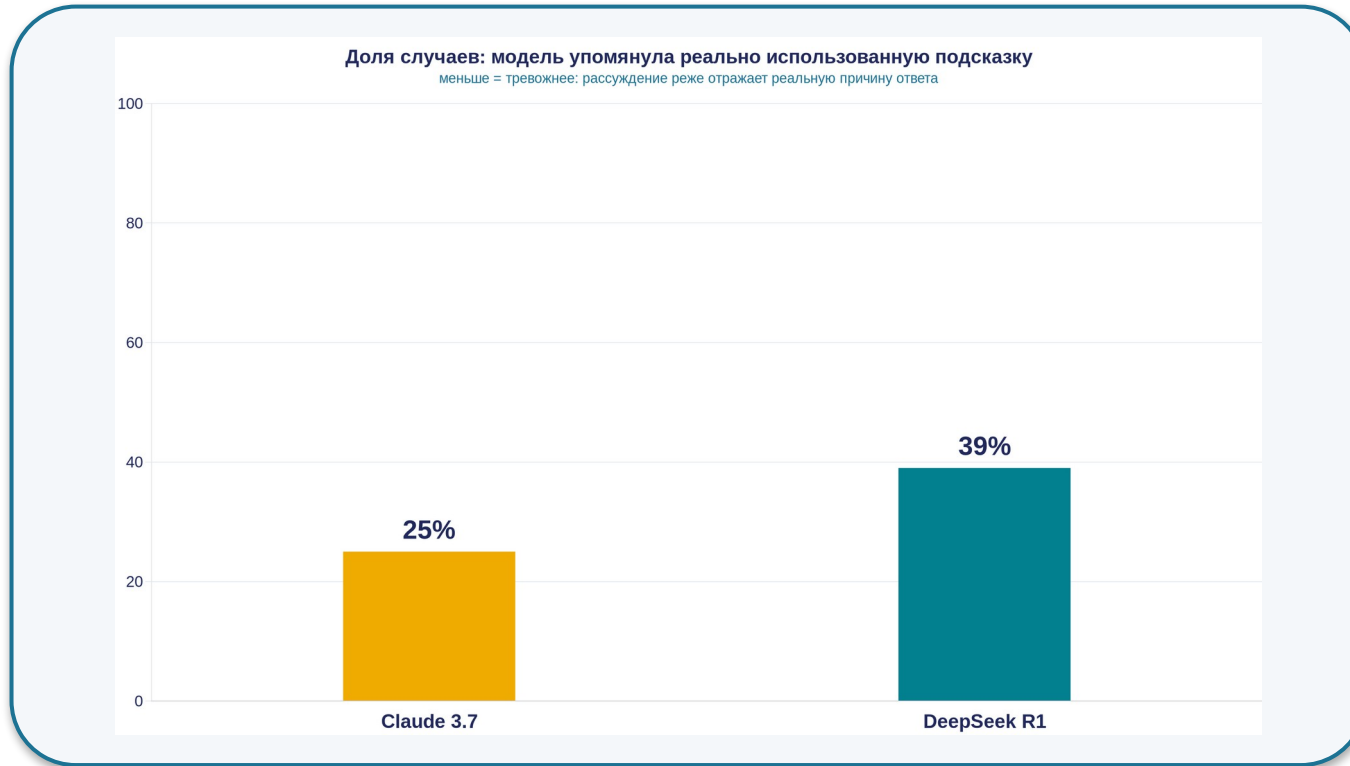
→ верно

CoT — инструмент под класс задач, не глобальный тумблер. Когда НЕ нужен: прямое извлечение факта, простая классификация — удлинняет ответ (дороже, медленнее) и иногда уводит в правдоподобную ошибку.

Поможет ли CoT именно вашей задаче — или там ответ извлекается напрямую?

Предел CoT: текст $\neq$ мысль.

Faithfulness (верность рассуждения) — степень, в которой проговорённая цепочка отражает реальные факторы, повлиявшие на ответ.

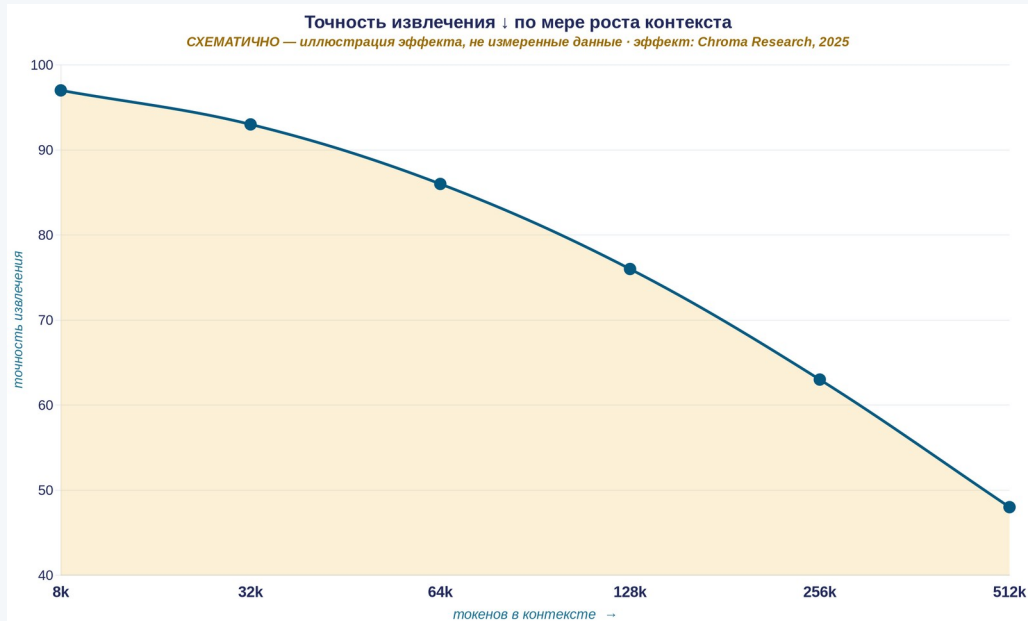


Эксперимент: задачу дали дважды — без подсказки и с подсказкой, меняющей ответ. Когда подсказка изменила ответ — упомянула ли её модель?

Контроль на самообъяснении модели — не контроль. Человек проверяет результат и факты против внешнего источника, а не правдоподобность текста. Верность падает на трудных задачах.

Контекст-инжиниринг: минимум высокосигнального.

Промпт-инжиниринг — одна инструкция. Контекст-инжиниринг — курирование всего набора токенов, видимых модели на инференсе.



context rot = тот же «lost in the middle» из Л2 §3 — новый термин, не новая сущность

Когда НЕ RAG (точка 1)

малый стабильный корпус, влезает в окно → full-context + кэширование префикса, а не RAG-инфраструктура



RAG здесь добавил бы хрупкость без выигрыша

«Найти наименьший набор высокосигнальных токенов, максимизирующий вероятность желаемого исхода» — это инженерное требование, не эстетика.

РАЗДЕЛ 2

RAG: поиск-дополненная генерация

Извлечь релевантное → положить в контекст → ответить с опорой на источник



Раздел 0
Открытие

Раздел 1
Промпт

Раздел 2
RAG

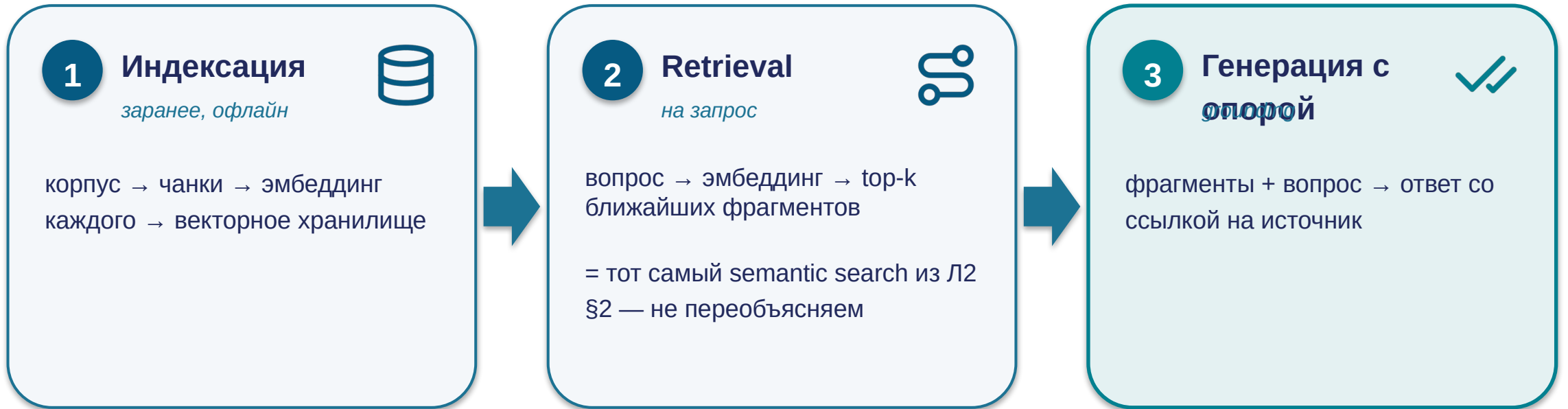
Раздел 3
Fine-tune

Раздел 4
API · агенты

Раздел 5
Фреймворк

Принцип RAG — три шага.

RAG = индексация → retrieval (semantic search Л2) → генерация с опорой; «не знаю» — корректный ответ.



«Не знаю» / «см. источник X» — корректный ответ RAG-системы. Правдоподобный ответ при нерелевантном retrieval — это дефект, а не «лучше, чем ничего».

Когда RAG — правильный выбор.

RAG оправдан, когда 4 признака складываются ОДНОВРЕМЕННО (конъюнкция, не дизъюнкция).

Большое / растущее

не влезает в окно целиком,
или дорого класть весь корпус
в каждый запрос

И

Меняется

документы, цены, регламенты
обновляются чаще, чем
выходят версии модели

И

Свежесть + провенанс

ответ опирается на
проверяемый источник; можно
показать, откуда факт

И

Приватная база

знания компании не входят в
веса публичной модели

Все 4 ОДНОВРЕМЕННО → RAG

Корп. база из тысяч регламентов, обновляется еженедельно, ответ с обязательной ссылкой на пункт: все 4 признака ✓ → образцовый профиль RAG.



Один признак сам по себе RAG не оправдывает — нужна конъюнкция.

Когда RAG — НЕ правильный выбор.

Знать, когда RAG не нужен, ценнее: это модная архитектура, её ставят туда, где она вредит.



1. Корпус влезает в окно

*ориентир — менее ~200k токенов,
меняется редко*

→ full-context + кэширование
префикса, не RAG-инфраструктура



2. Отдать фиксированную политику / значение

тариф, цена, пункт регламента, правило

→ детерминированный lookup /
статическая страница



3. Нет наблюдения за retrieval

observability отсутствует

→ скрытая бомба: не падает с
ошибкой, а уверенно отвечает
неправильно — отложенный
невидимый отказ

RAG оправдан только когда выполнены все три условия: (а) проще механизмы не справляются, (б) нужна генерация, а не фиксированное значение, (в) есть процесс измерения качества retrieval.

Провал RAG на масштабе.

«Вернул что-то» ≠ «вернул правильное». У RAG нет сигнала «не нашёл» — он всегда отдаёт к ближайших, даже нерелевантных.



Legal-AI

«ближайшие» дела из другой юрисдикции / отменённый прецедент — модель опирается на них как на факт



Medical-RAG

смешал фрагменты разных пациентов — близки по симптомам, клинически объединять нельзя



Support-бот

работал на сотнях статей; после роста до тысяч качество тихо просело — никто не заметил

Air Canada revisited

Диагноз

сгенерированный правдоподобный текст поставлен в роль, требовавшую извлечённого проверенного факта — отказ grounding

Правильная альтернатива

фиксированная политика → lookup / страница; нужен диалог → RAG со strict grounding, обязательная цитата, явное «не знаю», human-review

РАЗДЕЛ 3

Fine-tune vs промпт vs RAG

Проблему знания мы решили через RAG. А если проблема не в знании, а в поведении модели — её тоном, форматом, политикой?



Раздел 0
Открытие

Раздел 1
Промпт

Раздел 2
RAG

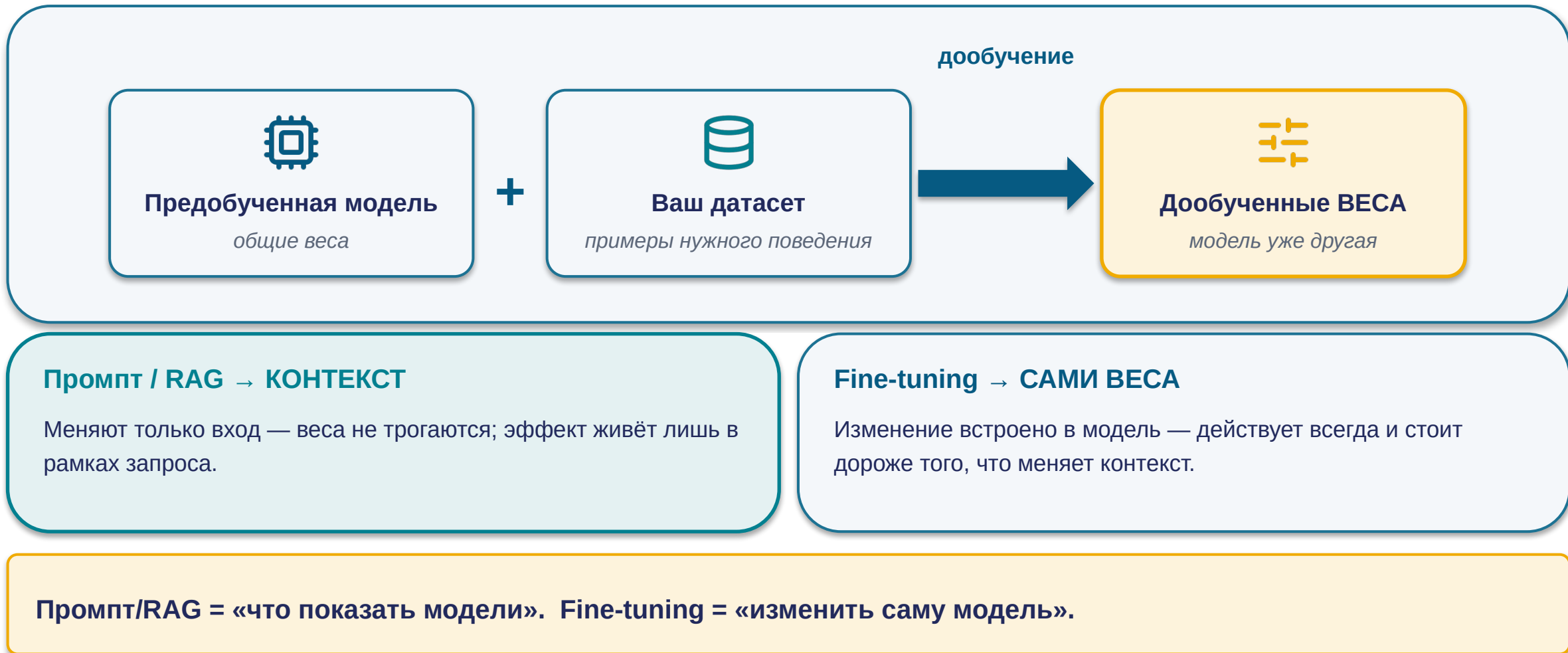
Раздел 3
Fine-tune

Раздел 4
API · агенты

Раздел 5
Фреймворк

Что такое fine-tuning.

Fine-tuning (дообучение) — продолжение обучения уже готовой модели на ваших данных. В Л1 — тип использования; здесь — архитектурный выбор, одна из ступеней лестницы.



Fine-tuning не умер — он сузился.

Fine-tuning (дообучение) — доп. обучение готовой модели на своих данных, меняются веса. В Л1 — тип использования; здесь — архитектурный выбор, одна из ступеней лестницы.



Ушло из fine-tuning (в 2026)

→ знание, факты, то, что меняется → RAG / длинный контекст

Защитить факты в веса: нельзя процитировать, нельзя обновить точно, нельзя удалить один



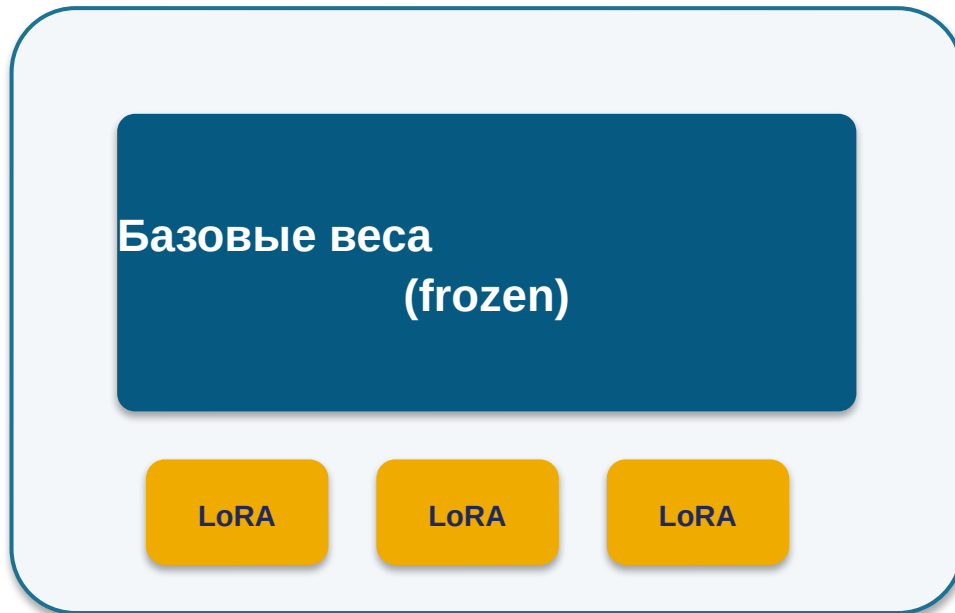
Осталось за fine-tuning

поведение · стиль · формат вывода · следование политике · дистилляция (меньшая модель учится у большей)

Реальный вопрос 2026 — не «RAG или fine-tuning», а «что здесь знание (→ RAG), что поведение (→ FT)». FT сузился, а не исчез.

PEFT вместо full fine-tuning.

PEFT — базовые веса замораживаются, обучается лишь небольшой набор адаптеров. LoRA — низкоранговые матрицы-адаптеры; QLoRA — то же поверх квантованной модели.



адаптеры — мегабайты vs гигабайты

1. Дешевле и быстрее

обучаются миллионы параметров вместо миллиардов; QLoRA — на одном GPU

2. Модульность

адаптеры мегабайты vs гигабайты; одна база — много специализаций

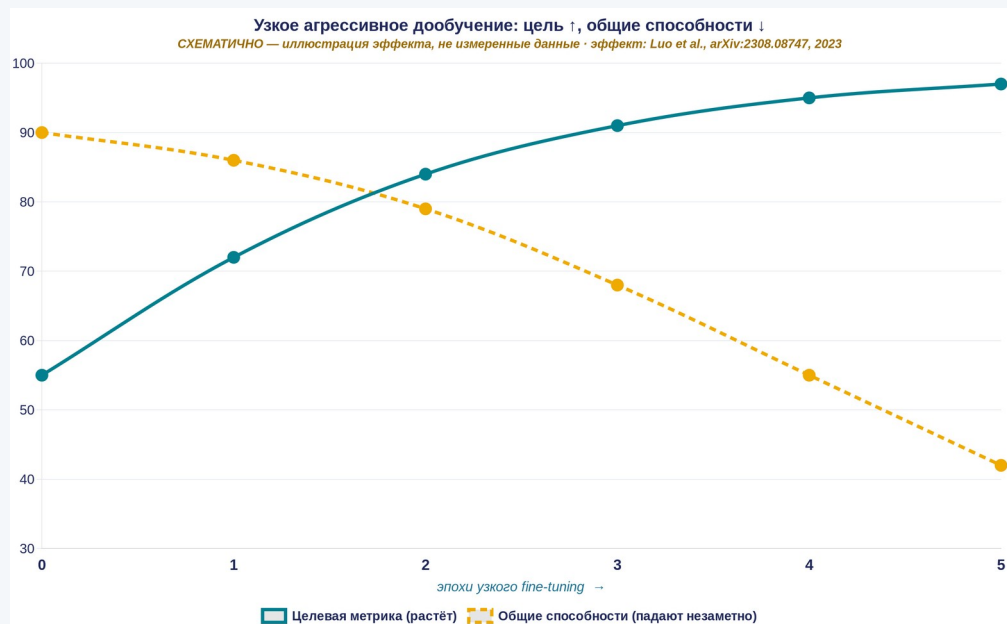
3. ↓ Риск catastrophic forgetting

база заморожена, физически не переписывается под новый сигнал — архитектурный аргумент

PEFT (LoRA/QLoRA) почти всегда лучше full fine-tuning: дешевле, модульнее и заметно снижает риск catastrophic forgetting — full FT в 2026 почти никогда.

Провал: catastrophic forgetting.

Catastrophic forgetting (катастрофическое забывание) — деградация общих способностей модели в результате узкого агрессивного дообучения.



тяжелее с ростом масштаба модели — у крупной выше исходный уровень, ей «больше падать»

Нет eval-петли на общих задачах + нет версий датасета/весов → не увидишь поломку до прода + не сможешь откатиться → это не «риск», это критерий «НЕ делай fine-tuning»

Правильно: PEFT (замороженные веса — ниже риск); для меняющегося знания — RAG, не FT вовсе.

Эмпирически наблюдается при continual fine-tuning (Luo et al. 2023). Механизмы — «исследования показывают» (preprint).

Критерии: что куда.

Знание меняется → RAG; поведение/тон/формат → PEFT; детерминированное → код. Гибрид — норма 2026.

Если задача требует...	→ правильный инструмент	...и НЕ этот, потому что
знание меняется / нужны свежесть, провенанс	RAG (или длинный контекст для малого корпуса)	не fine-tuning: устареет, дорого переобучать, риск forgetting
стабильное поведение / тон / формат / политика	fine-tuning (PEFT)	не RAG: подаёт знание, но не меняет манеру модели
снизить стоимость / латентность на узкой задаче	дистилляция (fine-tuning)	не RAG/промпт: не уменьшают размер модели
детерминированный, верифицируемый ответ	обычный код, без ИИ	ни RAG, ни FT: ИИ добавит недетерминизм без выигрыша

Гибрид — норма 2026 там, где у задачи **ОДНОВРЕМЕННО** есть и проблема знания, и проблема поведения. Не «больше компонентов лучше» — знание и поведение разнесены по правильным механизмам.

РАЗДЕЛ 4

API · tools · MCP · агенты + безопасность

От собеседника в окне чата — к компоненту продакшен-системы. И кто видит данные в цепочке.



Раздел 0
Открытие

Раздел 1
Промпт

Раздел 2
RAG

Раздел 3
Fine-tune

Раздел 4
API · агенты

Раздел 5
Фреймворк

API-слой: модель как компонент системы.

3 механизма делают модель: встраиваемой, активной, экономически жизнеспособной.



Structured output

модель обязана вернуть ответ строго по схеме (JSON), а не свободный текст

→ **выход = валидные данные, не текст для парсинга**

встраиваемая



Function calling

модель возвращает «вызови инструмент X с аргументами Y»; исполняет ваш код, не модель

→ **модель активная в системе**

активная



Prompt caching

переиспользовать вычисленное состояние неизменного префикса

→ **не переплачивать за повторяемую часть**

экономически жизнеспособная

Ни один из трёх не делает модель надёжнее или аудируемое — они расширяют, что модель может, не меняя её недетерминированной природы. Правило лестницы не отменяется.

MCP — стандарт подключения инструментов.

MCP (Model Context Protocol) — открытый стандарт: единый способ подключать инструменты, данные и контекст к LLM. «USB-C для инструментов LLM».



$N \times M$ несовместимых интеграций → **$N + M$** через MCP

инструмент один раз = MCP-сервер; модель один раз = MCP-клиент

Мини-таймлайн принятия:



Anthropic
11/2024



OpenAI
03/2025



Google
04/2025



независимый фонд
→

Стандартизация подключения ≠ безопасность подключаемого — и усугубляет проблему доверия.

- код в вашем окружении / доступ к данным
- описание попадает в контекст модели — носитель prompt injection (инъекция в промпт; разбор — s25)
- ещё одна граница доверия и retention-политика

Удобство подключения — не аргумент за подключение. Аргумент — требование задачи и приемлемость новой границы доверия.

Принятие/масштаб экосистемы MCP — перепроверить ко дню лекции.

Цикл агента: plan → act → check → iterate.

Агент — архитектура, где модель не делает один проход, а работает в цикле, сама определяя последовательность шагов.



Паттерны цикла (ReAct, Reflexion, Plan-and-Execute) — в главе. Проектировать агента = проектировать защиту на каждом из 4 шагов.

Workflow vs Agent.

Предсказуемая задача → workflow; непредсказуемая И ценность оправдывает кратный рост → агент.



Workflow (рабочий поток)

LLM и инструменты по предопределённым в коде путям

- последовательность шагов известна заранее
- предсказуемо, аудируемо
- большинство надёжных продакшн-систем — это workflow



Агент

LLM динамически определяет собственный процесс

- последовательность заранее не зафиксирована
- кратно больше токенов, чем чат
- ниже аудируемость, выше риск петель

Могу ли я заранее, до запуска, выписать последовательность шагов? **да (даже с ветвлениями) → workflow** ·
принципиально нет И ценность оправдывает кратные стоимость/риск → агент

Найди простейшее решение. «Лень формализовать» не делает задачу непредсказуемой.

Мульти-агент по умолчанию — не апгрейд (дебат — в главе)

Провалы агентов.

Каждый провал — режим отказа цикла агента в датированном кейсе: урок + альтернатива.



1. Петля без лимитов

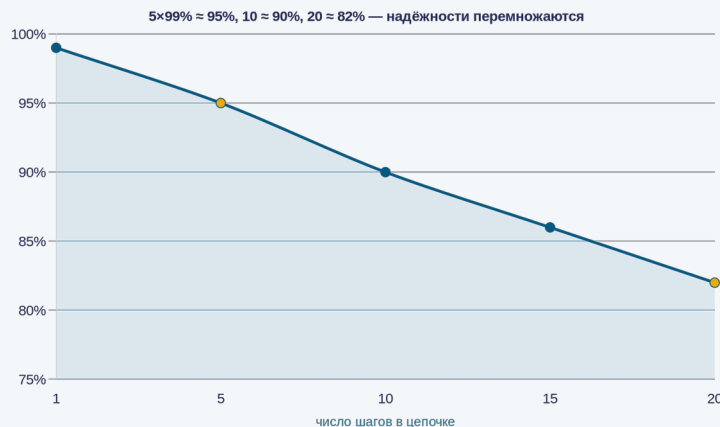
Агент на «синхронизируй заказы» получил HTTP 429 → план → вызов → 429 → ...

\$4 200 за 63 часа

Почему: агент не «видит» ни стоимости, ни времени, ни повторяемости

Правильно: задача предсказуема → `retry-with-backoff` скрипт, не агент; `budget/loop`-лимиты ВНЕ агента

2. Reliability compounding



Вывод: «улучшить шаг» — слабый рычаг; «меньше хопов + валидация между шагами» — сильный



3. Мульти-агентная хрупкость

зависимые подзадачи → параллельные субагенты принимают конфликтующие неявные решения

Правильно: **single-threaded** линейный агент; мульти-агент — только широко-параллельное независимое

РАЗДЕЛ 4 · БЕЗОПАСНОСТЬ

Кто видит данные в цепочке

Разобрали, надёжен ли агент. Теперь — другой вопрос: агент действует через инструменты и внешние API, так кто видит ваши данные и кто может прислать команду?



Раздел 0
Открытие

Раздел 1
Промпт

Раздел 2
RAG

Раздел 3
Fine-tune

Раздел 4
API · агенты

Раздел 5
Фреймворк

Кто видит данные в цепочке.

Каждое звено — своя граница доверия и своя retention-политика. Данные за периметром живут не по вашим правилам.



Судебный приказ переживает вашу политику

суд может обязать хранить логи поверх договорных «30 дней» (NYT v. OpenAI)



ZDR покрывает не всё

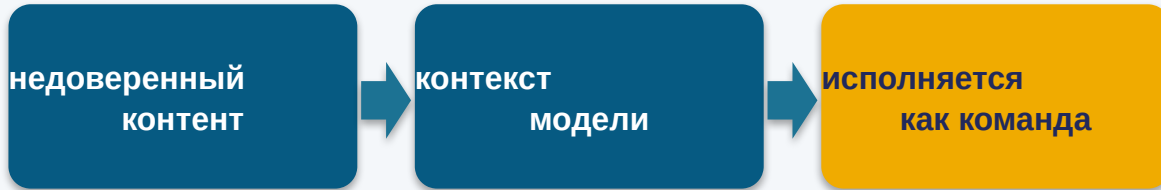
вне ZDR: third-party, Files, batch, MCP — а агент это и есть third-party

«У нас ZDR» ≠ «нигде ничего не хранится». Контрмера — карта данных по каждой фиче, не галочка. Регулируемое — не слать без ZDR/BAA.

ZDR (Zero Data Retention) — провайдер не хранит содержимое; least-privilege — минимум прав. BAA — договор об обработке. Хронология retention — в главе.

Атаки через инструменты.

Prompt injection (инъекция в промпт) — текст во внешних данных модель интерпретирует как инструкцию. Модель не отличает «данные» от «команды».



GitHub MCP heist

issue с инструкцией + переизбыточный токен → ассистент выгрузил приватные репозитории

Катастрофа = инъекция × широкие права. Убери любой из двух — атака не проходит.

4 правила проектирования

- 1 Least-privilege**
минимально необходимые токены/доступы
- 2 Изоляция недоверенного контента**
отдельно от привилегий
- 3 Human-in-the-loop на write**
необратимое только через подтверждение человека
- 4 Allowlist / pin**
только аудированные; фиксировать версии/хеши; deny-by-default

Tool poisoning / rug-pull и полная CVE-хронология MCP-инцидентов — в главе/notes, не на видимом слое.

РАЗДЕЛ 5

Как выбрать: фреймворк решения

Мы разобрали все архитектуры по отдельности — и где каждая проваливается. Теперь соберём это в один инструмент выбора.



Раздел 0
Открытие

Раздел 1
Промпт

Раздел 2
RAG

Раздел 3
Fine-tune

Раздел 4
API · агенты

Раздел 5
Фреймворк

Лестница архитектурной сложности.

Оставайся на нижней ступени; поднимайся только под требование задачи.



Нижняя ступень — «обычный код без ИИ»: лестница начинается с вопроса «а нужен ли ИИ вообще».

Матрица выбора.

Матрица — структура для аргумента, не сумма баллов. Заливка: золото = сильная сторона, бирюза = слабая.

Критерий ↓	 Промпт	 RAG	 Fine-tune	 Агент	 Код (без ИИ)
Знание большое / меняется	мало	да	нет	через RAG	—
Свежесть / провенанс	нет	да + цитата	нет	да	детерм.
Стоимость	низкая	средняя	высокая разово	кратно выше	минимум
Аудируемость	средняя	высокая	низкая	низкая	полная
Недетерминизм / риск петель	низкий	низкий	низкий	высокий	нет



Детерминированная, верифицируемая, повторяемая задача → обычный код, без ИИ. ИИ добавил бы лишь недетерминизм + стоимость + латентность + поверхность для prompt injection.

Чек-лист «прежде чем строить».

- 1 Решается обычным кодом (детерм., верифиц.)? → не добавляй ИИ
- 2 Хватит одного промпта (+few-shot/CoT)? → остановись здесь
- 3 Знание меняется / нужны свежесть, провенанс? → RAG, не FT
- 4 Нужно стабильное поведение / тон / формат? → fine-tuning (PEFT)
- 5 Корпус влезает в окно и стабилен? → длинный контекст + кэш
- 6 Предсказуема → workflow. Непредсказуема и ценна → агент+лимиты
- 7 Подзадачи независимы и параллельны? → мульти-агент; иначе линейный
- 8 Данные чувствительны? → карта данных + least-privilege + ZDR/BAA

Worked example (задача А)

«2000 регламентов, обновляются еженедельно, ответ со ссылкой на пункт» → шаги 3+8 → RAG со strict grounding. Причины: ось «частота изменений» + ось «провенанс»

Mini-apply (задача В)

«бот техподдержки на ~150 коротких FAQ, меняются раз в квартал, ответы из утверждённых формулировок» — пройдите чек-лист сами за 2 минуты

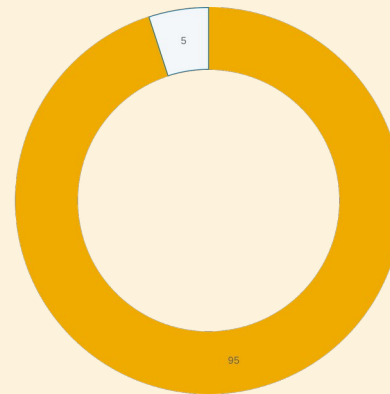
Сначала попытка — потом сверка. Сформулируйте ответ (архитектура + ≥ 2 причины по осям + ≥ 1 условие смены) ДО разбора.

Человек-валидатор + урок MIT NANDA.



Агент делает — человек проверяет результат и факты

- против независимого источника истины
- на значимых решениях — ДО того, как станет действием
- НЕ «убедительно ли звучит рассуждение» (callback s07: self-rationale ≠ контроль)
- это шаг check цикла агента + пункт 8 чек-листа + human-in-the-loop на write



~95%

корпоративных GenAI-пилотов без измеримого ROI — корень в learning gap и провале интеграции, не в качестве модели

«Запустить ИИ» ≠ «получить ценность». Решает архитектурно-интеграционная дисциплина. Иногда правильный ответ — простейшая архитектура или не-ИИ.

AI-архитектура — несущая ось отраслевых лекций.



Модуль 1 закрыт: Л1 (что такое AI) → Л2 (почему промпт работает) → Л3 (как собрать систему и КАК ВЫБРАТЬ)

Дальше — отраслевые Лекции 4–17: тот же вопрос в новой обёртке — «какая архитектура и почему именно она, и при каком условии ответ был бы другим»



Задание — Семинар 3

«Архитектурный выбор: чат / агент / RAG / API — 3 кейса»

Применить восьмишаговый чек-лист к 3 кейсам; для каждого: архитектура + ≥ 2 причины по осям матрицы + ≥ 1 условие смены выбора + критерий «здесь это было бы НЕ так». Mini-apply задача B (s28) — разминка перед семинаром.

Аппарат для центрального вопроса собран — прогоняйте чек-лист на каждой отраслевой лекции.

Вопросы

Спасибо за внимание

Семинар 3 «Архитектурный выбор» — на следующей неделе. Дополнительные вопросы — на e-mail.